

«Сравнение и анализ оптимизаций компиляторов»

Отчёт по учебной практике

Современные компиляторы проводят большое количество оптимизаций для написанного кода. Их понимание позволит лучше понять, как именно происходит трансляция кода в инструкции процессора, что нужно для написания более эффективных и безопасных программ. Для сравнения были выбраны 3 следующих компилятора:

- CLang (версия 22.1.1)
- MSVC (версия 19.50.35726)
- Intel oneAPI DPC++/C++ Compiler (версия 2025.3.2) (далее — ICX)

Характеристики системы, на которой проводились замеры времени работы:

- Процессор: AMD Ryzen 7730U with Radeon Graphics
- ОЗУ: 16 GB, 3200 MHz
- ОС: Windows 11 Pro 23H2
- x86-64 архитектура

Самые “интересные” качества компилятора — это, прежде всего, скорость работы программы и её размер. Поэтому были проведены замеры на специальной программе-тесте. Она содержит арифметические действия, циклы, записи в массивы и прочее, что может быть оптимизировано. Ниже приведены параметры компиляции, используемые в замерах.

CLang:

- clang -O0 optbench.c -o nooptimization.exe (без оптимизаций)
- clang -O2 optbench.c -o speedopt.exe (скорость работы)
- clang -Os optbench.c -o sizeopt.exe (размер файла)

MSVC:

- cl /Od optbench.c (без оптимизаций)
- cl /O2 optbench.c (скорость работы)
- cl /O1 optbench.c (размер файла)

ICX:

- icx /Od optbench.c (без оптимизаций)
- icx /O2 optbench.c (скорость работы)
- icx /Os optbench.c (размер файла)

Проанализировав результаты замеров можно сделать вывод, что компилятор CLang лучше всего оптимизирует код. Однако это не совсем верно.

В исходном тесте присутствовало большое количество “printf”, что негативно влияло на замеры, так как эта функция заставляет программу делать системные вызовы, а также работать с выводом. В итоге получается, что компиляторы MSVC и ICX провели оптимизации, но это практически никак не повлияло на замеры по времени. Исправив эту проблему, получаем аналогичные результаты с CLang.

Ещё один момент, который стоит отметить — это разный вывод программы, скомпилированной с оптимизациями и без. Это проявляется у компиляторов CLang и ICX. Вывод без оптимизации: **“220”** ; с оптимизациями: **“2201223Common subexpression elimination000100007100072000730007400074000740007400074000740007000000”**.

Причина этого — undefined behavior в коде, вызванном делением на ноль. Без оптимизаций программа “падает” на этом участке кода, в то время как с оптимизациями она старается продолжить выполнение. По стандарту C/C++ компилятор имеет право предполагать, что в корректной программе неопределенное поведение не наступает, поэтому он всячески преобразовывает код, чтобы избежать UB.

Рассмотрев оптимизации компиляторов можно сделать следующие выводы: все три компилятора успешно справляются с базовыми оптимизациями: разложение и свёртка констант, арифметические тождества, лишние присваивания и тому подобное. Однако между ними есть различия.

ICX в среднем демонстрирует наиболее продвинутые техники оптимизации, ориентированные на максимальную скорость выполнения кода, часто в ущерб его размеру.

- Активно заменяет дорогие операции (деление `idiv`, умножение) на дешёвые (сдвиги `shl`, умножение на магические константы `imul`)
- Полностью разворачивает циклы, превращая их в линейный код, что устраняет накладные расходы на проверку условий, но сильно увеличивает размер бинарника
- Лучшим образом вычисляет константы на этапе компиляции, избавляясь от лишних вычислений во время выполнения

CLang во многом похож на ICX в базовых оптимизациях. Он также заменяет операции на более дешёвые (пример с умножением) и эффективно использует регистры, минимизируя обращения к памяти. Главное отличие между ними — это то, что CLang оставляет циклы, в то время как ICX полностью их разворачивает. CLang — это по сути золотая середина между производительностью и размером кода.

MSVC часто генерирует код, который легче отлаживать, но который может уступать в производительности.

- Включает проверки переполнения стека (`__report_rangecheckfailure`), а также инструкции `int 3` и чаще использует безопасные, но более медленные инструкции
- Чаще обращается к памяти (`dword ptr [...]`) даже тогда, когда данные можно было бы держать в регистрах (в отличие от ICX)
- Реже применяет полное развертывание циклов или сложную подстановку констант, оставляя больше вычислений на этап выполнения
- В некоторых примерах использует прямое деление (`idiv`), которое медленнее эмуляции через умножение